

PRAKTIKUM INFORMATIK 2

M. Farmbauer, M. Niemetz

Templates und Exceptions

Wärmespeicherkomplettierung

Mai 2026, Version: 4.11

1 Einführung

In dieser Praxiseinheit wird die Wärmespeicher-Anlage fertiggestellt und die Funktion der Anlagensimulation abgeschlossen.

Neben den in den letzten Teilen erstellten Klassen wird zusätzlich eine Klasse Pipeline entwickelt, deren Objekte es gestatten, das Medium – und damit die darin gespeicherte Wärmemenge – von einem Kessel in einen anderen Kessel zu transferieren.

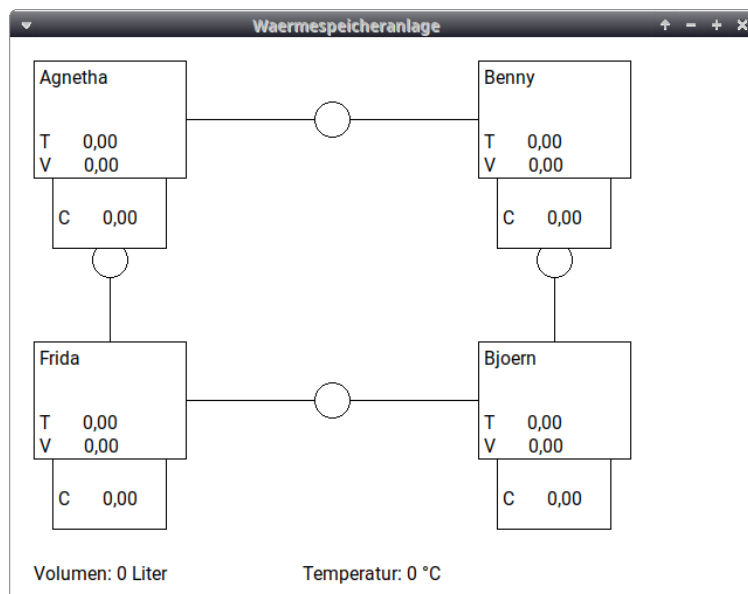


Abbildung 1: Beispielabbildung für eine komplette Energiespeicheranlage mit Kesseln, Brennern und Pipelines nach Fertigstellung dieser Praktikumsaufgabe

Eine solche Pipeline verbindet genau zwei Kessel: Die bisher voneinander völlig unabhängigen Kessel werden also nun wie in Abbildung 1 gezeigt mit Pipelines verbunden. Die Pipelines sind mit Pumpen versehen und können so das Medium vom einen in den anderen Kessel transportieren.

Das Konzept der Containerklasse aus der vorangehenden Aufgabe wird zur Verwaltung der Objekte dieser neuen Pipeline-Klasse mit Hilfe des C++ Template-Mechanismus „generalisiert“.

Außerdem setzen Sie in diesem Teil auch zum ersten Mal den Exception-Mechanismus ein, um Fehler bzw. vom normalen Programmablauf abweichende Ausnahmesituationen zu handhaben.

Vorbereitung

- ☐ ☕☕☕ **Template-Mechanismus:** Finden Sie heraus, was Templates sind, wozu man sie einsetzt und wie man mit diesem Mechanismus umgeht – sowohl bei der Definition wie auch bei der Verwendung. 8

- ☐ ☕☕ **Machen Sie sich mit der Funktionsweise des Exception-Mechanismus vertraut.:** Klären Sie hierbei insbesondere die unterschiedliche Bedeutung von **throw** und **catch** sowie die typischen Anwendungsfälle für den Einsatz des Exception-mechanismus ab. 11
- ☐ ☕☕ **Abgeleitete Klassen:** Lesen Sie nach, wie Sie in C++ eine Klasse von einer bereits bestehenden Basisklasse ableiten können und was das bedeutet. 12
- ☐ ☕ **Umwandlung von **string**-Objekten in C-Zeichenketten:** Recherchieren Sie das Verhalten der Methode **c_str** der Klasse **string**. 13
- ☐ ☕ **Lokale **static**-Variablen:** Lesen Sie nach, wie sich das Verhalten von lokalen **static**-Variablen von anderen lokalen Variablen unterscheidet. 13
- ☐ ☕ **mutable-Elemente in Klassen:** Lesen Sie nach, wie sich das Verhalten von **mutable**-Datenelementen in Klassen von anderen Datenelementen unterscheidet. 13

2 Projekteinrichtung

Bevor mit der weiteren Entwicklungsarbeit begonnen werden kann, merken Sie sich den aktuellen Zustand Ihres Entwicklungsprojekts.



Sichern Sie den aktuellen Entwicklungszustand wieder durch Setzen eines „Tags“.



Stellen Sie sicher, dass der aktuelle Entwicklungszustand mit einem Commit gesichert wurde. Wenn nicht, holen Sie dies bitte **unbedingt** nach.



Erzeugen Sie für die aktuelle Version ein neues „Tag“, indem Sie in **gitk** einen Rechtsklick auf die aktuelle Version ausführen und **Markierung erstellen** auswählen. Verwenden Sie **Komponentenintegration_fertiggestellt** als Name der Markierung.

Alternativ können Sie natürlich auch einfach in der **git-bash** das Kommando

```
git tag "Komponentenintegration_fertiggestellt"
```

ausführen.

Laden Sie Ihr Projekt auf den Server, und aktivieren Sie dann in **git gui** die Option „Markierungen übertragen“ bzw. alternativ bei Verwendung der Kommandozeile

```
git push --tags
```

Prüfen Sie, ob das „Tag“ korrekt in der Liste der Tags in Ihrem Projekt auf dem gitlab-Server auftaucht.

3 Klasse Boiler um eine Medien-Schnittstelle erweitern

Boiler
+ ContentTemperature :double + ContainedVolume :double + Position :Point + Dimension :Point + Name :string – BBurner :Burner*
+ Boiler(Name :string Position :Point Dimension :Point ContainedVolume :double, ContentTemperature :double) + Boiler(Source :Boiler&) + ~Boiler() + addContent(MediaTemperature :double, MediaAmount :double) :void + addHeat(Amount :double) :void + show() :void + contains() :bool + activateHeating() :void + isHeatingAvailable() :bool + operator=(rhs :?) : Boiler + extractMedium(AmountRequested : double, AmountDelivered : double&, Temperature : double&) : void

Die Klasse **Boiler** benötigt eine Schnittstelle, um das Arbeitsmedium ganz oder teilweise aus dem Kessel entfernen zu können. Die hierfür vorgesehene Funktion **extractMedium** soll dabei überprüfen, wie viel Flüssigkeit wirklich im Kessel vorhanden ist, und **die Entnahmemenge ggf. entsprechend beschränken**.

Diese Methode der Fehlerbehandlung wird in der Softwaretechnik häufig angewendet, um die Zusammenarbeit zwischen verschiedenen Softwareteilen effizient zu gestalten, aber dennoch eine korrekte Funktion sicherzustellen.

Der Nachteil dieser Vorgehensweise ist, dass der aufrufende Softwareteil die korrigierte Menge beachten und ggf. darauf mit einer angepassten weiteren Vorgehensweise darauf reagieren muss.

Um dies umzusetzen, wird die Information, welche Menge tatsächlich entnommen wurde, an die aufrufende Funktion übermittelt, damit diese bestimmen kann, welche Menge aus dem Kessel erhalten wurde.

Für die aufrufende Funktion ist neben der tatsächlich entnommenen Menge aber auch die Temperatur der Flüssigkeit entscheidend, da diese Information gebraucht wird, um die entnommene Flüssigkeitsmenge in einen anderen Kessel einzufüllen (siehe vorangehende Aufgaben).

Da eine Funktion nur maximal einen Rückgabewert besitzen kann, verwenden Sie zwei zusätzliche Parameter, welche Sie per Referenz (und nicht per Wert) übergeben.

Hinweis: Eine Übergabe per Referenz kann sowohl über ein Pointer-Argument, wie auch durch ein Referenz-Argument realisiert werden. Entscheiden Sie selbst, welche Lösung Sie bevorzugen – und begründen Sie Ihre Entscheidung!

Damit besitzt die Methode **extractMedium** insgesamt **drei Argumente**, wobei das erste Argument die vom Aufrufer angeforderte Entnahmemenge (**double**-Wert) angibt, während die beiden anderen Argumente die tatsächlich entnommene Menge bzw. die Temperatur des entnommenen Mediums (beides **double**-Werte) an den Aufrufer zurücktransportieren



Erweitern Sie die Klasse **Boiler** um eine Schnittstelle zum Entnehmen von Arbeitsmedium.

Testen Sie nun Ihre Implementierung, indem Sie Ihr Programm compilieren.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

4 Die neue Klasse Pipeline

Pipeline
– Source : Boiler* – Target : Boiler* – PumpSize : int
+ Pipeline(Source :Boiler*, Target :Boiler*, PumpSize :int) + transferMedium(Amount :double) :void + show() :void + contains(Clickposition :Point&) :bool

Eine Pipeline verbindet zwei Kessel miteinander und ermöglicht es, die Flüssigkeit mit Hilfe einer in die Pipeline eingebauten Pumpe aus dem Quellkessel in den Zielkessel zu befördern.

Erzeugen Sie die neue Klasse, indem Sie die Dateien Pipeline.h und Pipeline.cpp erzeugen, und die Typdeklaration in die h.-Datei und die Implementierungen der Funktionen in die .cpp-Datei schreiben.

Wichtig: Beachten Sie, dass die neue .h-Datei einen Schutz gegen Mehrfachinklusion benötigt.

Die Klasse Pipeline besteht dabei aus folgenden Datenelementen:

Name	Typ	Beschreibung
Source	Boiler*	Zeiger auf das Quellobjekt, also den Kessel, aus dem die Flüssigkeit abgepumpt werden soll.
Target	Boiler*	Zeiger auf das Zielobjekt, also den Kessel, in den die Flüssigkeit gefüllt werden soll.
PumpSize	int	Durchmesser des Kreises, der die Pumpe der Pipeline auf dem Bildschirm repräsentiert.

und Funktionen (Details dazu in den folgenden Unterabschnitten!):



Definieren Sie eine Klasse Pipeline welche den Transport von Flüssigkeit zwischen den Kesseln ermöglicht.

Name	Typ	Parameter und Beschreibung
<i>Konstruktor</i>	—	Der Konstruktor hat die Parameter <code>Source</code> und <code>Target</code> (beide vom Typ <code>Boiler*</code>), welche Pointer auf die beiden <code>Boiler</code> -Objekte darstellen, die mit der Pipeline verbunden werden. Zusätzlich existiert auch noch der optionale Parameter <code>PumpSize</code> des Typs <code>int</code> , der den Durchmesser der Pipelinepumpe auf dem Bildschirm angibt. Alle Parameter sollen in den entsprechenden Attributen gespeichert werden.
<code>transferMedium</code>	<code>void</code>	Die Methode <code>transferMedium</code> besitzt einen Parameter <code>Amount</code> (des Typs <code>double</code>) und transportiert die durch <code>Amount</code> gegebene Menge an Flüssigkeit vom Quellkessel der Pipeline zum Zielkessel.
<code>show</code>	<code>void</code>	Die Methode <code>show</code> besitzt keine Parameter und zeichnet die Pipeline als Linie zwischen den Mittelpunkten von Quell- und Zielkessel mit einem Kreis in der Mitte der Linie um die Pumpeneinheit zu symbolisieren.
<code>contains</code>	<code>bool</code>	Liefert zurück ob die per <code>Point</code> -Parameter übergebene Position auf dem Bildschirm innerhalb des Kreises der Pumpe liegt oder nicht.

Definieren Sie die Klasse `Pipeline` entsprechend in der `.h`-Datei und implementieren Sie den **Konstruktor** in der `.cpp`-Datei.

Hinweis: Informationen zur Implementierung der übrigen Methoden finden Sie in den folgenden Abschnitten

Testen Sie nun, ob ihr Projekt fehlerfrei übersetzbar ist.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

4.1 Methode `transferMedium`

Zur Entnahme aus dem Quellkessel wird die oben beschriebene Methode `extractMedium` des von `Source` referenzierten Kessels verwendet.

Die von dieser Methode gelieferte Information über die entnommene Menge bzw. die Temperatur des Mediums wird dann der Methode `addContent` des von `Target` referenzierten Zielkessels übergeben.



Implementieren Sie die Methode `transferMedium` der Klasse `Pipeline`

4.2 Methode show



Implementieren Sie die Methode `show` der Klasse `Pipeline`

Die Methode `show()` zeichnet die Pipeline als einfache Linie zwischen den Mittelpunkten von Quell- und Zielkessel.

Auf halben Weg zwischen Quelle und Ziel wird zusätzlich ein Kreis auf die Linie gezeichnet, der die Pumpe der Pipeline symbolisiert. Das Ergebnis kann man gut in Abbildung 1 erkennen.

Um den Code übersichtlich zu halten, sollten Sie jeweils die Koordinaten der beiden Anfangspunkte, der Linien sowie des Mittelpunktes des Kreises in Variablen zwischenspeichern. Dafür legen Sie geeignete (private) Datenelemente in Ihrer Klasse an und verwenden am besten auch eine private Funktion für die Aktualisierung dieser Werte (das ist später noch nützlich!).

Wichtig: Damit man eine saubere Darstellung auf dem Bildschirm erhält, müssen die `show`-Methoden der Pipelines zeitlich **vor** den `show`-Methoden der Kessel und Brenner ausgeführt werden.

Hinweis: Zum Zeichnen der Linie verwendet man `Line` und zum Zeichnen des Kreises `EllipticalArc`. Eine ausführliche Beschreibung der beiden Funktionen findet man in der Datei `WinAdapt.pdf`.

Prüfen Sie nun, ob Ihr Projekt fehlerfrei kompilierbar ist.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

4.3 Mausbedienung mit der Methode contains



Erweitern Sie die Klasse `Pipeline` um die Bedienbarkeit per Mausklick.

Wie bereits bei den Klassen `Boiler` und `Burner` soll auch bei den die Pumpen der Klasse `Pipeline` eine Maussteuerung ermöglicht werden.

Realisieren Sie eine Methode `contains` für die Klasse `Pipeline`, die `true` zurückliefert, falls die Maus innerhalb des Kreises, der die Pumpe der Pipeline repräsentiert, geklickt wurde, und `false`, falls der Punkt außerhalb liegt.

Da die Pumpen als Kreis dargestellt werden, genügt es nicht, wenn die Koordinaten der Maus beim Klicken innerhalb des die Pumpe umschließenden Rechtecks liegen. Die Koordinaten müssen innerhalb es **Kreises** der Pumpe liegen.

Mit Hilfe des Satzes von Pythagoras lässt sich aber leicht berechnen, ob die Koordinaten des Klicks $\vec{P}_{\text{Klick}}(x_K, y_K)$ innerhalb des Kreises um den Mittelpunkt $\vec{P}_{\text{Pumpe}}(x_P, y_P)$ liegen:

$$(x_K - x_P)^2 + (y_K - y_P)^2 < r^2 \quad (1)$$

In Vektorschreibweise würde man dies so formulieren:

$$\left| \vec{P}_{\text{Klick}} - \vec{P}_{\text{Pumpe}} \right| < r \quad (2)$$

Natürlich wird diese Berechnung durch die Klasse `Point` durch geeignete Funktionen unterstützt, so dass Sie den Code Ihrer Funktion deutlich eleganter und übersichtlicher schreiben können:

Hierfür müssen Sie zunächst den Differenzvektor zwischen dem Mittelpunkt der Pumpe und dem angeklickten Ort auf dem Bildschirm ermitteln und dann prüfen ob dessen Länge kleiner ist als der Radius der Pumpe. Dafür stellt `Point` neben einer Differenz zweier `Point`-Objekte auch einen Vergleichsoperator zwischen `Point` und `double` zur Verfügung.

Prüfen Sie, ob Ihr Projekt ohne Fehler compilierbar ist.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

5 Dynamischer Test der Pipeline-Klasse im Verbund

Setzen Sie nun in der Datei `Stages.h` den Wert von `STAGE` auf `CREATE_PIPELINES` und compilieren und starten Sie Ihr Programm. Die Ausgabe sollte aussehen wie in Abbildung 2 dargestellt.

Prüfen Sie, ob alles erwartungsgemäß funktioniert und beheben Sie jetzt ggf. vorhandene Fehler.

Testen Sie insbesondere die in dieser Aufgabe neu hinzugekommene Bedienung der Pumpen in den Pipelines über das Anklicken der Pumpen mit der linken Maustaste.

Die aus früheren Tests bekannten Tastatureingaben sowie der Mausklick zur Aktivierung der Kesselheizung stehen bei diesem Test nicht zur Verfügung.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.



Testen Sie Ihre Implementierung der Pipeline im Modus `STAGE CREATE_PIPELINES`.

6 Containertemplate erzeugen

Für die Verwaltung der `Boiler`-Objekte wurde in der vorangehenden Übungseinheit eine Containerklasse `StorageComponentContainer` erzeugt, welche die Verwaltung der Kessel in der Anlage übernommen hat.

Recherche ☕☕☕

Template-Mechanismus



Erzeugen Sie mit Hilfe eines Templates eine generische Containerklasse, welche die Verwaltung von Pipelines (und anderen Objekten) erlaubt.

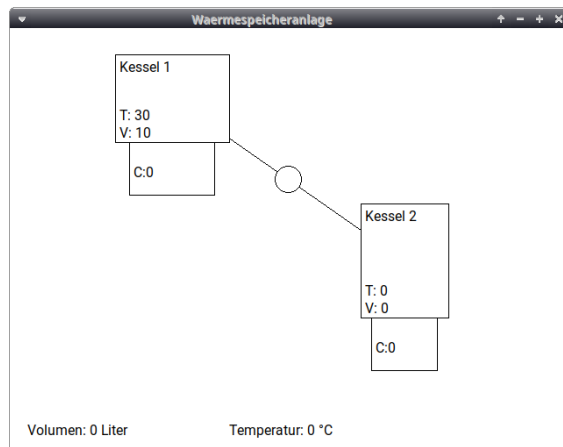


Abbildung 2: Darstellung der Testanordnung mit Pipeline direkt nach dem Programmstart.

Für die Pipelines benötigen wir nun ebenfalls eine entsprechende Containerklasse. Dazu könnte man nun den `StorageComponentContainer` einfach kopieren, und in der Implementierung dabei überall den Wortteil `Burner` durch `Pipeline` bzw. `Kessel` ersetzen ...

... aber das kann es ja nicht sein. Man stelle sich nur vor, die Containerklasse soll um eine zusätzliche Methode erweitert werden oder ein Fehler muss behoben werden. Mit wenig Vorstellungskraft zeichnen sich endlose Möglichkeiten für Fehlersuche und -behebung ab.

Es wäre doch schön, wenn dieses fehlerträchtige Kopieren und Ersetzen eleganter gelöst werden könnte, z.B. indem man der Klasse den Typ, den sie verwaltet, als eine Art Parameter übergibt. Natürlich ist das mit den bisherigen Mechanismen nicht möglich, da Typen zur Compiler-Laufzeit wichtig sind, Funktionsparameter aber erst zur Laufzeit übergeben werden.

Tatsächlich ist das gewünschte Verhalten dennoch realisierbar, und zwar mit Hilfe eines neuen Mechanismus in C++, den sogenannten **Templates**. Diese Templates stellen insgesamt ein sehr mächtiges Werkzeug der Programmierung in C++ dar. Sie werden im Folgenden aber nur im einfachst möglichen Fall verwendet.

6.1 Entwicklung eines Templates basierend auf `StorageComponentContainer`

Setzen Sie zunächst **vor** die vorhandene **Klassendefinition** der bereits in der vorangehenden Aufgabe zur Verwaltung der Kessel-Objekte entwickelten Klasse `StorageComponentContainer` und der zugehörigen Iteratorklasse `StorageComponentContainerIterator` sowie vor **alle Methodenimplementierungen außerhalb einer Klassendefinition** jeweils die Zeile

template <typename Elementtype>

Diese Zeile informiert den Compiler darüber, dass die folgende Definition ein Template darstellt und in diesem Template das Wort `Elementtype` einen Parameter repräsentiert, der einen Typ enthält.

Nun nehmen Sie folgende weitere Änderungen an der Datei `StorageComponentContainer.h` vor:

- Ersetzen Sie jede Verwendung des Typs `Boiler` durch die Verwendung des Parameters `Elementtype`.
- Falls in Ihrer `storagecomponentcontainer.h`-Datei noch irgendwo das Wort `Boiler` vorkommt, ist das unerwünscht, da wir ja jetzt eine Klasse entwickeln, die unabhängig vom Brenner funktionieren soll. Finden Sie also ggf. bessere Namen.
- Ersetzen Sie in den Funktionsköpfen der Implementierungen der Methoden jedes Vorkommen des Namespace-Operators `StorageComponentContainer::` durch die mit dem neuen Template-Parameter erweiterte Form des Namespace-Operators `StorageComponentContainer<Elementtype>::`
- Ersetzen Sie auch in den Funktionsköpfen der Implementierungen der Methoden jedes Vorkommen des Namespace-Operators `StorageComponentContainerIterator::` entsprechend durch `StorageComponentContainerIterator<Elementtype>::`
- Ersetzen Sie jedes Auftreten des Typs `StorageComponentContainerIterator` durch `StorageComponentContainerIterator<Elementtype>`
 - außer in der Klassendefinition selbst.
- Ersetzen Sie jedes Auftreten des Typs `StorageComponentContainer` durch `StorageComponentContainer<Elementtype>`
 - außer in der Klassendefinition selbst.

Nun ist Ihr neues Klassen-Template schon funktionsfähig. Verwendet wird es zum Beispiel folgendermaßen:

```
StorageComponentContainer<Pipeline> PipelineList;
```

Dabei gibt der Typ in spitzen Klammern den Typ an, der anstelle des Parameters für die Erzeugung des Objekts verwendet werden soll.

Testen Sie nun ob Ihr Projekt korrekt übersetzt werden kann und beheben Sie eventuelle Fehler.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

Damit haben wir genau das erreicht, was wir uns eingangs erhofft hatten: Der Energiespeichercontainer kann nun zur Verwaltung beliebiger anderer Klassen verwendet werden.

Taste	Aktion
ctrl a ... ctrl d	Aktivieren der Heizung 1-4
a ... d	Befüllen Kessel 1... 4 nach Vorwahl
1 ... 9	Vorwahl des Volumens
A ... Z	Vorwahl der Temperatur

Tabelle 1: Tastaturbelegung, die für den Testbetrieb im aktuellen Modus `STAGE == CREATE_TEMPLATE` gültig ist.

7 Test der Gesamtanlage

Ändern Sie nun in `Stages.h` die Definition der Präprozessorkonstante `STAGE` auf `STAGE CREATE_TEMPLATE` ab und testen Sie Ihre neuen Implementierungen und beheben Sie eventuelle Fehler. Die Ausgabe auf dem Bildschirm sollte aussehen, wie in Abbildung 1 dargestellt.

Sie können für den Test die bereits bekannten Tastaturkommandos aus Tabelle 1 einsetzen.

Darüber hinaus sollten Sie auch die Bedienung der Brenner und Pumpen mit der Maus testen. Diese sollten aktiviert werden, wenn Sie auf den Kessel oder den Brenner klicken.

Analysieren Sie den durch `STAGE == CREATE_TEMPLATE` aktivierten Testcode in `main.cpp` und klären Sie eventuelle Fragen.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

8 Exception-Handling

Falls Sie, wie in den vergangenen Aufgaben vorgeschlagen, bereits einmal untersucht haben, was passiert, wenn Sie einem leeren Kessel Wärme zuführen, so werden Sie bemerkt haben, dass der Kessel daraufhin eine Temperatur von „unendlich“ (INF) erreicht. Wenn Sie dann mit diesem Wert weiter rechnen erhalten Sie schnell ungültige Werte (NaN: not a number).

Dies ist natürlich physikalischer Unsinn, da das Kesselmaterial lange vor dem Erreichen so hoher Temperaturen zerstört wird. Daher wird im weiteren Verlauf der Aufgabe der Kessel mit einem Schutz gegen Überhitzung ausgestattet.

In der Vorlesung haben Sie vermutlich inzwischen bereits das Konzept des Exception-Mechanismus kennengelernt. Falls nicht (oder falls Ihnen

?

Testen Sie die neu entwickelten Funktionen im Modus `CREATE_TEMPLATE` und beheben Sie eventuelle Fehler.



Erweitern Sie Ihre Wärmespeicheranlage mit einer Schutzfunktion gegen das Schmelzen überhitzter Kessel.

Recherche ☕☕

Machen Sie sich mit der Funktionsweise des Exception-Mechanismus vertraut.

die Details entfallen sind), lesen Sie bitte in Ihrem C++-Buch bzw. dem Skript nach oder fragen Sie Ihre Dozenten in der Lehrveranstaltung.

8.1 Entwickeln einer Exception-Klasse

Definieren Sie in der Datei `Boiler.h` eine neue Klasse mit dem Namen `ExceptionBoilerOverheating`, welche von der Klasse `std::exception` abgeleitet ist, die sie über Inkudieren des Headers `stdexcept` einbinden können.

Diese neue Klasse soll folgende Attribute und Methoden erhalten:

Name	Typ	Beschreibung
<code>BoilerTemperature</code>	<code>double</code>	Temperatur des überhitzenden Kessels vor der problematischen Aktion
<code>BoilerContent</code>	<code>double</code>	Aktueller Inhalt des überhitzenden Kessels
<code>AddedHeat</code>	<code>double</code>	Wärmemenge, die im Überhitzungsmoment zugeführt werden sollte
<code>FailingBoiler</code>	<code>Boiler*</code>	Zeiger auf den Kessel, welcher überhitzen würde

und Methoden:

Name	Typ	Parameter und Beschreibung
<i>Konstruktor</i>	—	Der Konstruktor hat die Parameter <code>FailingBoiler</code> vom Typ <code>Boiler*</code> , <code>BoilerTemperature</code> , <code>BoilerContent</code> und <code>AddedHeat</code> (alle vom Typ <code>double</code>), die den Attributen der Klasse entsprechen. Alle Parameter sollen vom Konstruktor in den entsprechenden Attributen gespeichert werden.
<code>what</code>	<code>const char*</code>	Die Methode <code>what</code> überlädt die von <code>std::exception</code> geerbte Methode und soll einen sinnvollen Beschreibungstext für das aufgetretene Problem liefern. (siehe dazu auch unten)

Beachten Sie, dass die Methode `what` eine `const` Methode sein muss, die zusätzlich auch noch die Eigenschaft `noexcept` tragen muss. Daher lautet der Prototyp der Methode:

```
const char* what() const noexcept;
```



Entwickeln Sie eine Exception-Klasse welche den für den Überhitzungsfall wichtigen Kontext transportieren kann.

Recherche

Abgeleitete Klassen

Der von `what` zurückgelieferte Text soll insbesondere die Werte der drei `double`-Attribute und den Namen des betreffenden Kessels enthalten.

Hinweis: Um den von `what` gelieferten Text zu konstruieren, müssen Sie sich Gedanken machen, wie, wo und wann Sie welche Teile der Lösung implementieren.

Mit Hilfe von `stringstream`-Objekten können Sie die Eigenschaften von `ostream`-Objekten mit denen von `string` Objekten kombinieren. Damit ist ein `stringstream`-Objekt gewissermaßen die Entsprechung in der Objektwelt zum bereits von C bekannten `sprintf`.

Insbesondere können Sie alle für `ostream`-Objekte existierenden Ausgabeoperatoren damit auf Strings als Ausgabemedium anwenden.

Hinweis:

- Den String eines `stringstream`-Objekts erhalten Sie über die Methode `str` als `string`-Objekt (Kopie der Originaldaten) geliefert.
- Objekte der Klasse `string` erlauben es, über die Methode `c_str` den String in eine C-Zeichenkette umzuwandeln, wie sie für die Textausgabe bei WinAdapt bzw. für den Rückgabewert von `what` benötigt wird. Diese Zeichenkette lebt aber ohne weitere Maßnahmen nicht länger als das `string`-Objekt.
- Achten Sie auf die Lebensdauer, der von `what` zurückgelieferten Zeichenkette. Diese muss die Ausführungsdauer der Funktion `what` übersteigen.

Hinweis: Um die Lebensdauer der Zeichenkette hinreichend lang zu gestalten, kann sie nicht einfach aus einer lokalen Variable der Funktion `what` gewonnen werden.

Es bieten sich mehrere funktionierende Lösungen an:

- Verwendung eines lokalen `static`-Objekts.
- Verwendung eines zusätzlichen Datenelements in der Klasse, welches das `string`-Objekt oder die Zeichenkette speichert und das im Konstruktor der Exception-Klasse beschrieben wird (da `what` ja eine `const`-Funktion ist).
- Verwendung eines zusätzlichen Datenelements in der Klasse, welches das `string`-Objekt speichert und das `mutable` ist, um in `what` modifiziert werden zu können.
- Zusätzliches Pointer-Element in der Klasse, welches auf einen externen Datenbereich bzw. ein `string`-Objekt zeigt, das in bekannter Weise im Konstruktor angelegt, im Destruktor gelöscht und in `what` modifiziert wird.

Frage: Machen Sie sich die Funktionsweise der verschiedenen Lösungen klar. Überlegen Sie sich dann die Vor- und Nachteile der unterschiedlichen Lösungen und entscheiden Sie sich für eine. Begründen Sie Ihre Entscheidung.

Recherche ☕

Umwandlung von `string`-Objekten in C-Zeichenketten

Recherche ☕

Lokale `static`-Variablen

Recherche ☕

`mutable`-Elemente in Klassen

Implementieren Sie nun die Methoden in der Datei `Boiler.cpp` und prüfen Sie, ob Ihr Projekt fehlerfrei kompiliert wird.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

8.2 Entdecken und Melden des Fehlers

Das Keramikmaterial, aus welchem Ihre Kessel gefertigt sind, beginnt bei einer Temperatur von ca. 3000°C zu schmelzen. Da nur in der Methode `addHeat` des Kessels solche Temperaturen erreicht werden können, fügen Sie in der Implementierung der Methode eine entsprechende Abfrage ein. Falls die Grenztemperatur überschritten werden würde, werfen Sie eine Exception des im vorangehenden Abschnitt definierten Typs `ExceptionBoilerOverheating`.

Achten Sie dabei darauf, bei der Erzeugung des geworfenen Elements dem Konstruktor von `ExceptionBoilerOverheating` die korrekten Werte **vor** der drohenden Überhitzung mitzuteilen und führen Sie in diesem Fall **keine** Veränderung am Kesselzustand durch.



Detektieren Sie das Überhitzen des Kessels und melden Sie das Problem durch Werfen einer Exception.

8.3 Verarbeitung des Fehlers

Es ist etwas unschön, dass unsere Simulation der Wärmespeicheranlage beendet werden würde, nur weil versehentlich ein Kessel überhitzt. Um dies zu verhindern, muss die Exception nun noch verarbeitet und der Fehler behandelt werden um eine Fortsetzung zu erlauben.

Sowohl in der Mausabfrage, wie auch in der Tastenabfrage in Ihrem Hauptprogramm werden die Brenner aktiviert. Daher ist an beiden Stellen das Einfügen von `try-catch` Blöcken nötig, welche gezielt die `ExceptionBoilerOverheating`-Objekte fangen. Als Fehlerverarbeitung sollen einfach die von `what` gelieferten Informationen auf `cout` ausgegeben werden.

Der entsprechende Code ist bereit sin Ihrem programm integriert. Sie können ihn durch eine Änderung in `Stages.h` aktivieren, in dem Sie `ERROR_HANDLING` auf 1 setzen.

Frage: Überlegen Sie sich, warum die `try-catch` Blöcke sinnvollerweise in der Maus- bzw. Tastenabfrage im Hauptprogramm stehen sollten und nicht z.B. in der `addHeat`-Methode der Klasse `Boiler` oder der Methode `feedFuel` der Klasse `Burner`.

Dadurch, dass die Exception gefangen wird und kein expliziter Programmabbruch implementiert wird, läuft das Programm nun bei Überhitzungsfehlern einfach weiter, wobei die Überhitzung verhindert und das Problem in der Konsole sichtbar gemacht wird.



Ergänzen Sie Ihr Hauptprogramm um eine Fehlerbehandlung um so ein unerwartetes Beenden des Programms zu verhindern.

Testen Sie nun Ihre Energiespeicheranlage und prüfen Sie, ob die Brenner im Überhitzungsfall Brennstoff verbrauchen oder nicht. Falls ja, ändern Sie die Methode `feedFuel` der Brenner so ab, dass der Zähler für den verbrauchten Brennstoff erst **nach** dem Aufruf der Methode `addHeat` des entsprechenden Kessels aktualisiert wird.

Frage: Überlegen Sie sich, warum die Reihenfolge der Aktionen in `feedFuel` wichtig ist und das Verhalten des Programms beeinflusst.



Erzeugen Sie nach dem erfolgreichen Test eine neue Revision und laden Sie das Ergebnis auf den Kursserver hoch.

Damit ist unsere Simulation einer Wärmeenergiespeicheranlage abgeschlossen.